

Redis + BullMQ Interview Preparation Guide (Tanglish)

Topics Covered

1. Caching
2. Rate Limiting
3. Distributed Locking
4. BullMQ Queues
5. TTL
6. Session Management
7. Pub/Sub
8. Cache Invalidation

1. CACHING

What is Caching?

Frequently use panna data ah:

temporary ah Redis la store pannradhu

Purpose:

- Database load reduce pannradhu
- API response speed improve pannradhu
- Frequently accessed data fast ah serve pannradhu

Real Flow

Without Cache:

Client
↓
API

↓
MongoDB

Every request:

DB hit aagum

With Cache:

```
Client
  ↓
API
  ↓
Redis Check
  ↓
If Cache Exists → Return Data
Else → Fetch DB → Store Redis
```

Node.js Example

```
router.get("/users", async (req, res) => {

  const cachedUsers = await redis.get("users");

  if (cachedUsers) {

    return res.json({
      source: "redis",
      data: JSON.parse(cachedUsers)
    });

  }

  const users = await userModel.find();

  await redis.set(
    "users",
    JSON.stringify(users),
    "EX",
    60
```

```
);  
  
return res.json({  
  source: "database",  
  data: users  
});  
  
});
```

Important Redis Commands

Command	Purpose
GET	read cache
SET	store cache
EX	expiry
TTL	remaining time
DEL	remove cache

Interview Answer

"Redis caching use panni repeated database queries reduce pannuvom and API response speed improve pannuvom."

Real Usage

- Product listing
 - User profile
 - Dashboard APIs
 - Frequently accessed APIs
-
-

2. RATE LIMITING

What is Rate Limiting?

Specific time la:

```
evlo requests allow pannanum
```

nu control pannradhu.

Why Needed?

Prevent:

- API abuse
 - brute force attacks
 - spam requests
 - server overload
-

Real Flow

```
Request
↓
Redis Counter Increase
↓
Check Limit
↓
Allowed / Blocked
```

Node.js Example

```
const ratelimiter = async (
  req,
  res,
  next
) => {
```

```
const ip = req.ip;

const key = `rate_limit:${ip}`;

const requests =
  await redis.incr(key);

if (requests === 1) {

  await redis.expire(
    key,
    60
  );

}

if (requests > 5) {

  return res.status(429).json({

    success: false,

    message:
      "Too many requests"

  });

}

next();

};
```

Important Concept

```
redis.incr()
```

automatically:

```
counter increase pannum
```

Example:

```
1
2
3
4
```

Interview Answer

"Redis rate limiting use panni user request count track pannuvom and limit exceed aana requests block pannuvom."

Real Usage

- Login APIs
 - OTP APIs
 - Payment APIs
 - Public APIs
-

3. DISTRIBUTED LOCKING

What is Distributed Locking?

```
Only one request certain operation execute panna allow pannradhu
```

Why Needed?

Prevent:

- duplicate payments
- duplicate orders
- race conditions
- stock issues

Problem Without Lock

Request A
Request B

same time execute aagum.

Result:

Duplicate processing

Redis Lock Command

SET order_lock locked EX 10 NX

Breakdown

Part	Meaning
SET	create key
order_lock	lock key
locked	value
EX 10	expire after 10 sec
NX	only if not exists

Why NX Important?

Already lock irundha:

new lock create pannaadhu

So:

only one request gets lock

Node.js Example

```
const lock = await redis.set(
  "order_lock",
  "locked",
  "EX",
  10,
  "NX"
);
if (!lock) {
  return res.status(409).json({
    success: false,
    message:
      "Another request is processing"
  });
}
try {
  console.log("Processing Order");
} finally {
  await redis.del("order_lock");
}
```

Interview Answer

"Distributed locking use panni concurrent requests control pannuvom and duplicate processing avoid pannuvom."

Real Usage

- Payment systems
 - Inventory management
 - Quotation ID generation
 - Order systems
-
-

4. BULLMQ QUEUES

What is BullMQ?

Redis-based background job queue system

Why Needed?

Heavy tasks:

- email sending
- PDF generation
- notifications

request-response cycle block panna koodadhu.

Architecture

```
Client
  ↓
API
```

```
↓  
BullMQ Queue  
↓  
Redis  
↓  
Worker  
↓  
Background Processing
```

Queue Example

```
import { Queue } from "bullmq";  
  
const emailQueue = new Queue(  
  "emailQueue",  
  {  
    connection: redis  
  }  
);
```

Add Job

```
await emailQueue.add(  
  "sendEmail",  
  {  
    email: "test@gmail.com"  
  }  
);
```

Worker Example

```
import { Worker } from "bullmq";

const worker = new Worker(

  "emailQueue",

  async (job) => {

    console.log(job.data);

  },

  {
    connection: redis
  }

);
```

Flow

```
API Request
  ↓
Queue.add()
  ↓
Redis stores job
  ↓
Worker picks job
  ↓
Process task
```

Important BullMQ Keys

Key	Purpose
completed	completed jobs
failed	failed jobs

Key	Purpose
events	queue events
stalled-check	stuck job detection

Interview Answer

"BullMQ use panni background processing implement pannuvom. Heavy tasks async ah worker process pannum so API response fast ah irukum."

Real Usage

- Emails
 - Notifications
 - Report generation
 - Video processing
-
-

5. TTL (Time To Live)

What is TTL?

Redis key evlo time survive aaganum nu define pannradhu

Why Needed?

- temporary cache
 - OTP expiration
 - session expiration
 - rate limiting reset
-

Example

```
await redis.set(  
    "otp",  
    "1234",  
    "EX",  
    60  
);
```

Meaning:

60 seconds ku apm delete aagum

TTL Check

TTL otp

Example Output:

45

Meaning:

45 seconds remaining

Interview Answer

"TTL use panni temporary Redis keys automatic ah expire panna mudiyum."

Real Usage

- OTP systems
- Cache expiry
- Session expiry

- Rate limiter reset
-

6. SESSION MANAGEMENT

What is Session Management?

User login state ah:

temporary ah maintain pannradhu

Traditional Problem

Server memory la sessions store panna:

- restart aana lose aagum
 - scalable illa
-

Redis Solution

Redis centralized session store ah use pannuvom.

Flow

```
User Login
  ↓
Session Created
  ↓
Redis Store Session
  ↓
User Requests
  ↓
Session Validate
```

Node.js Example

```
await redis.set(  
  `session:${userId}`,  
  JSON.stringify(user),  
  "EX",  
  3600  
);
```

Read Session

```
const session =  
  await redis.get(  
    `session:${userId}`  
  );
```

Interview Answer

"Redis session management use panni scalable centralized user session storage implement pannalam."

Real Usage

- User authentication
 - Login sessions
 - Access tokens
-
-

7. PUB/SUB

What is Pub/Sub?

Real-time messaging system

Meaning

Term	Meaning
Publisher	message send pannum
Subscriber	message receive pannum

Flow

```
Publisher
  ↓
Redis Channel
  ↓
Subscribers Receive Message
```

Publisher Example

```
await redis.publish(
  "chat",
  "Hello"
);
```


Subscriber Example

```
redis.subscribe("chat");

redis.on(
  "message",
  (channel, message) => {

    console.log(message);

  }
);
```

Interview Answer

“Redis Pub/Sub use panni real-time communication implement pannalam.”

Real Usage

- Chat applications
 - Live notifications
 - Real-time dashboards
 - Multiplayer games
-
-

8. CACHE INVALIDATION

What is Cache Invalidation?

Database update aana:

```
old cache remove pannradhu
```

Why Needed?

Old cache irundha:

```
stale data return aagum
```

Example Problem

DB la:

```
User name changed
```

But Redis cache:

```
old name store pannirukum
```

Solution

DB update aprm:

```
await redis.del("users");
```

Flow

```
Update Database
↓
Delete Cache
↓
Next Request
↓
Fresh Data Fetch
```

Interview Answer

"Cache invalidation use panni stale cache remove pannuvom so latest data serve aagum."

Real Usage

- Product updates
 - User profile updates
 - Dashboard refresh
-

MOST IMPORTANT REDIS COMMANDS

Command	Usage
SET	store value
GET	read value
DEL	delete key
INCR	increase counter
EXPIRE	set TTL
TTL	remaining time
KEYS *	list keys
MONITOR	live Redis operations
PUBLISH	send pub/sub message
SUBSCRIBE	receive messages

FINAL INTERVIEW SUMMARY

Redis

"Redis is an in-memory datastore used for caching, rate limiting, session management, distributed locking, Pub/Sub messaging, and background job processing."

FULL ARCHITECTURE YOU BUILT

```
Express API
  ↓
Redis Cache
  ↓
Rate Limiter
  ↓
Distributed Lock
  ↓
BullMQ Queue
  ↓
Worker Processing
  ↓
Pub/Sub / Sessions
```

INTERVIEW TIPS

- Explain with real examples
- Mention Redis is in-memory datastore
- Mention Redis is extremely fast
- Mention TTL for temporary data
- Mention BullMQ for background processing
- Mention distributed locking prevents race conditions
- Mention Pub/Sub for real-time communication